Beyond Exact Copies: Recovering the Lineage of Reused AI Agent Skills

Bhavesh Maurya
bmaurya@stevens.edu
Stevens Inst. of Tech.
Hoboken, NJ, USA

Abhishek Thakkar
athakkar1@stevens.edu
Stevens Inst. of Tech.
Hoboken, NJ, USA

Mahima Jain
mjain19@stevens.edu
Stevens Inst. of Tech.
Hoboken, NJ, USA

Pratiksha Vilas Pawar
ppawar6@stevens.edu
Stevens Inst. of Tech.
Hoboken, NJ, USA

1 INTRODUCTION

Coding agents such as Claude Code now take part in everyday software work: they edit files, run commands, and review changes on a developer's behalf. To make an agent follow a team's procedure for a recurring task, developers package that procedure as an *agent skill*. A skill is a folder whose `SKILL.md` file starts with YAML front matter, holding a `name` and a `description`, followed by instructions written in Markdown; the folder may also contain scripts and reference documents [1, 8]. The agent first reads only the descriptions of the available skills and loads a skill's full instructions when it judges that the current task matches one. Anthropic released the format as an open specification in October 2025, and nine months later millions of `SKILL.md` files were sitting in public GitHub repositories [8].

Skills spread differently from conventional dependencies. A library installed from npm or PyPI is declared in a manifest, pinned to a version, and updated through the package manager. A skill has no such channel. Developers copy its folder into their own repository, often edit it along the way, and nothing records where it came from. The GitSkills dataset shows how common this is: of 3,797,117 `SKILL.md` files collected from 282,200 repositories, 50.5% are byte-for-byte copies of another file in the dataset [8]. Copies that were edited after copying are not part of that figure, because any edit produces a new content hash.

We use the following terms throughout. An *exact copy* has the same content hash as another file. A *derived variant* is a skill produced by copying another skill and then editing it: renaming, trimming, extending, rewording, or translating it. Two skills copied from a common source are derived variants of each other. A *template sibling* shares only generic scaffolding with another skill, such as section headings, boilerplate guidance, or a catalog's house style, while its task-specific instructions were written separately. A *skill family* is a group of skills connected by derivation, and its *lineage* is the order in which its members appeared. These notions follow code-clone research, where clones range from identical fragments (Type-1) to fragments with renamed identifiers (Type-2) and with added, removed, or changed statements (Type-3) [13].

Lineage matters because an agent executes a skill instead of merely displaying it. A person reading an outdated README can notice the problem and work around it, whereas an agent acts on stale instructions directly [9]. Skills can also carry risk. Security audits have found vulnerable and malicious skills in public collections [11], and in January 2026 a single campaign planted more than 1,200 malicious skills in the OpenClaw skill marketplace [16]. Once a flawed skill has been copied and edited, its relatives are hard to find.

2 RELATED WORK

2.1 Reuse and adaptation of agent skills

Gao et al. [9] present the first study of skills as engineered artifacts. They collected 18,463 skills from the skills.sh registry and 23,199 skills from 5,876 GitHub repositories, and linked a repository skill to a registry skill when both carried the same YAML `name`, using a longest-common-subsequence similarity to choose among same-name candidates and to reject pairs below 0.1. This recovered 3,709 reuse links. Most reused skills turned out to be near-verbatim copies, 53% were never modified after adoption, and among reused skills that were never updated locally, 40.2% had a registry source that changed afterwards, changes the copies never received. The linkage only sees a copy whose source is published on skills.sh and that kept the source's name. The authors acknowledge that it misses renamed and heavily edited skills and assumes that reuse flows from the registry into repositories; 76% of the repository skills they collected stayed unlinked, and some of those may be undetected reuse.

Wu et al. [15] study how downstream developers adapt published skills, using public forks of six popular skill repositories as evidence. From 1,126 adaptation instances they build a taxonomy of 46 adaptation patterns in 13 families, and they report that 18.6% of adaptations add security-sensitive content to the instruction text. They call for tools that detect inconsistent modifications. Because the method relies on GitHub's fork relation, it observes only copies made by forking those six repositories. A skill pasted into an unrelated repository leaves no fork record.

2.2 Clone detection for agent skills

Zhu et al. [16] propose SkillReuse, the work closest to ours. It represents a skill's YAML metadata, prose, and code separately, combines the per-channel similarities with a flat TF-IDF similarity and structural features in a logistic regression, and reaches an F1 of 0.939 on SkillReuse-Bench, a set of 300 labelled pairs. Applied to 137,470 skills crawled from the SkillsMP marketplace, it places 66.8% of skills in at least one clone pair, most of them across authors. The

authors are explicit about what clone detection does not establish. Structural similarity is not derivation, so detected pairs may include convergent development; the clone relation is undirected and does not say which skill came first; and the analysis uses the `SKILL.md` file alone, without repository history. Two properties of the benchmark also matter for our problem. Half of its 150 positive pairs are synthetic mutations rather than real edits, and its hard negatives are same-category pairs whose similarity lies between 0.15 and 0.45. Highly similar skills that were written independently, such as two skills generated from one template, are therefore not represented. Finally, families formed as connected components of the clone graph chain together: the largest one contains 61,651 skills.

2.3 Code clones and reuse on GitHub

Clone detection for source code is a mature field. Roy et al. [13] define the standard clone types and compare detection techniques, and SourcererCC scales token-based detection of near-miss clones to large code corpora [14]. Lopes et al. [12] showed that file-level duplication is pervasive on GitHub. Kim et al. [10] introduced clone genealogies, which follow groups of clones across versions and separate consistent changes, applied to every member of a group, from inconsistent ones. These techniques were designed for source code, where tokens and syntax carry most of the evidence. Skills are mostly natural language and are often written from shared scaffolds, which makes surface similarity a weaker signal of shared origin.

3 PROBLEM STATEMENT

Developers reuse agent skills by copying a skill's folder into their own repository, often editing it on the way in. No registry entry, manifest, or version pin records this relationship, so once a skill is copied its lineage is lost. Three groups pay for this. Developers who adopt a skill are never told when its source is fixed or improved, and because agents execute skills, a stale copy keeps acting on outdated or unsafe instructions. Skill authors cannot see where their work has spread or how it was changed. Security reviewers who flag one harmful skill cannot locate its edited relatives.

Existing studies address this only in part. Approaches that trace reuse need an explicit record of it: either the copy still carries the name of a skill published on a public registry [9], or it lives in a repository created with GitHub's fork feature [15]. Copies that were renamed, heavily edited, or taken from an unregistered repository are invisible to both. Clone detection for skills [16] removes the need for such records, but it measures similarity, not origin. It cannot tell a derived variant from a template sibling, it does not order related skills in time, and it has not been evaluated on the highly similar but independently written pairs that templates produce.

The result is a gap in what we know. The only study that measured whether copies receive updates found that 40.2% of never-edited reused skills had missed later changes to their source [9], but that measurement covers only the registry-linked minority of skills. For skills shared by plain copying, which GitSkills shows to be widespread [8], we cannot yet say which skills derive from which, how concentrated reuse really is once edited copies are counted, or whether changes to an original ever reach its copies.

4 PROPOSED SOLUTION

We propose to recover skill lineage from the content of skills alone and then use it to study reuse. The solution has two parts.

First, we build a classifier that decides, for a pair of skills, whether one is a derived variant of the other, a template sibling, or unrelated. Separating these classes is the core difficulty. Skills are short, mostly written in English, and often scaffolded from the same templates, so two independently written skills can share much of their text while a reworded or translated copy can share little. No fixed similarity threshold separates these cases, and training a model on labels produced by such a threshold would only teach the threshold back to the model. We therefore build the ground truth by hand: team members independently label real candidate pairs from GitHub against a written rubric, and we measure their agreement. Because the pairs are sampled across similarity levels, the labelled set deliberately contains the highly similar template siblings that existing benchmarks leave out. Besides lexical similarity, the classifier uses signals aimed at the template problem, chiefly a similarity computed only over the lines a pair shares that do not recur across many unrelated skills, together with front-matter, structural, and repository evidence. We compare it with exact hashing, MinHash-style and TF-IDF similarity, and a re-implementation of SkillReuse [16].

Second, we apply the classifier to candidate pairs drawn from the GitSkills dataset [8], group skills into families, order family members by the first commit of each `SKILL.md` where that history is available, and measure how concentrated reuse is once edited copies are counted. In the next phase we will use these families to test whether upstream changes reach their copies, following the genealogy view of code clones [10].

The design answers each limitation identified above. It needs no registry entry and no fork record, which matters in practice: GitHub code search indexes a fork only when it has more stars than its parent, so only three repositories in GitSkills are forks [7]. It separates derivation from template similarity instead of treating every similar pair as a clone, and it tests that separation on real pairs rather than synthetic mutations. It uses commit history to give derived pairs a direction where possible. Finally, it forms families with community detection rather than plain connected components, so that a few bridging pairs cannot chain unrelated skills into one giant family.

5 DATASET EXTRACTION

Source. GitSkills is distributed as a single SQLite file with four tables: `artifacts`, with one row per file occurrence; `repos`, with repository metadata; `artifact_siblings`, with the files bundled with each skill; and `mining_runs`, with collection provenance [8]. For this phase we use the official sample released by the dataset authors [7]: the 13,000 distinct contents with the lowest content hashes, together with every row the full dataset holds about them, i.e., 29,786 file occurrences in 11,841 repositories owned by 10,786 accounts. Because content hashes are effectively uniform, the sample is a random sample of distinct skills, and because every content group is complete, its copy counts are exact. Queries written for the sample run unchanged on the full 41 GB file, which we will use in Phase 3.

Fields. Table 1 lists the fields we extract.

Table 1: GitSkills fields used in this study.

Field(s)	Use
`file_sha`, `repo_full_name`, `path`, `location_class`	occurrences, exact-copy counts
`filename`, `dedup_primary`, `frontmatter_valid`, `body_chars`	scope filtering
`content`, `name`, `description`	skill text (representatives)
`first_commit_at`	ordering derived pairs
`artifact_siblings.entry_name`	bundled-file similarity
owner (prefix of `repo_full_name`)	same-owner and cross-owner reuse

Table 2: Scope filters applied to the official GitSkills sample.

Filter	Distinct skills
All distinct contents	13,000
Exact basename `SKILL.md`	12,763
Valid YAML front matter	11,197
Body $\geq$ 200 characters (dataset field)	11,010
Body $\geq$ 200 characters after our own parsing	11,009

Scope. The filename search behind GitSkills also returns lowercase `skill.md` files and similar names that predate the format, so we keep only representatives whose exact basename is `SKILL.md`. We require valid front matter because the name and description are inputs to our classifier, and a body of at least 200 characters, which removes stubs and symbolic-link skills whose content is only a link target. Table 2 shows the effect of each filter.

The resulting working set has 11,009 distinct skills with 26,664 file occurrences in 10,770 repositories owned by 9,826 accounts. Commit history is available for 2,567 of these skills, and 3,763 bundle at least one other file. Already at this scale, 538 front-matter names are carried by two or more distinct contents.

Decomposition. For each representative we split the file into its name (normalised), description, prose (the body with fenced code removed), code blocks, and heading sequence, and attach repository context: the owners of all its occurrences, the name of the skill's folder, and the names of its bundled files. The result is stored as a small SQLite database with two tables, `skills` and `occurrences`, which the remaining steps read.

6 STUDY DESIGN

Figure 1 summarises our approach. It has five stages: the first two are automatic, the third is our manual analysis of the dataset, and the last two build the classifier and apply it.

Stage 1: Data preparation. We extract the working set described in Section 5 and split every skill into its name, description, prose, code blocks, and headings, together with its repository context.

Stage 2: Candidate pair generation. Comparing every pair of the 11,009 skills would take about 60 million comparisons, and more than a trillion on the full dataset. We therefore score only pairs proposed by at least one of three inexpensive blockers. First, MinHash locality-sensitive hashing [3] over word 5-gram shingles of the body

Table 3: Similarity bands of candidate pairs and of the labelled sample.

Band	Definition	Candidates	Labelled
B1	$J \geq 0.9$	426	60
B2	$0.7 \leq J < 0.9$	615	80
B3	$0.5 \leq J < 0.7$	1,278	80
B4	$0.3 \leq J < 0.5$	1,348	80
B5	same name, $J < 0.3$	1,889	50
B6	LSA neighbour, $J < 0.3$	1,901	30
B7	random pair (not a candidate)	–	20
B0	other LSH match, $J < 0.3$	628	0
	Total	8,085	400

(128 permutations) proposes pairs whose estimated Jaccard similarity is at least 0.3. Second, same-name blocking proposes distinct contents that carry the same normalised front-matter name, capped at 45 pairs per name so that common names such as `code-review` do not dominate. Third, nearest neighbours in a latent semantic analysis (LSA) space [6], built from TF-IDF vectors of the body reduced to 128 dimensions, propose each skill's three closest skills when their cosine is at least 0.9; this catches reworded copies that share few exact shingles. Together the blockers propose 8,085 candidate pairs, which we assign to bands by estimated Jaccard similarity J or by the blocker that found them (Table 3).

Stage 3: Manual analysis. We followed the three steps of explore, analyse, and synthesise.

Explore. We first read a random set of skills, the most-copied contents, and candidate pairs from every band to learn why skills resemble each other. **[State what you read, e.g., "each of us read 25 skills and 20 pairs".]** Much of the similarity in the middle bands turned out to come from shared scaffolding rather than copying. Examples include a catalog of machine-generated skills whose names follow the pattern `u02156-...` and whose texts differ in a few substituted words, integration skills for many different applications that wrap the same Rube MCP (Composio) boilerplate, and a series of German legal-practice skills that repeat the same checklist lines. Conversely, many same-name pairs, such as two skills called `code-review`, were written independently. These observations led to our three-class labelling scheme.

Analyse. We drew a stratified sample of 400 pairs across the bands of Table 3, including 20 random pairs that no blocker proposed as easy negatives. To keep the sample diverse, a skill appears in at most two pairs and a name in at most four. The left–right order of each pair is randomised, and annotators never see similarity scores or bands. Two team members labelled each pair independently in a side-by-side diff viewer, following a written rubric that adapts the criterion Gao et al. used to validate their reuse links [9]. A pair is *derived* when the skills share distinctive content that is unlikely to be written twice independently: verbatim or lightly reworded passages, or the same task-specific steps, examples, commands, paths, or configuration values; renamed and translated copies count as derived. A pair is a *template* pair when the overlap is limited to generic scaffolding while the task-specific instructions differ. Otherwise

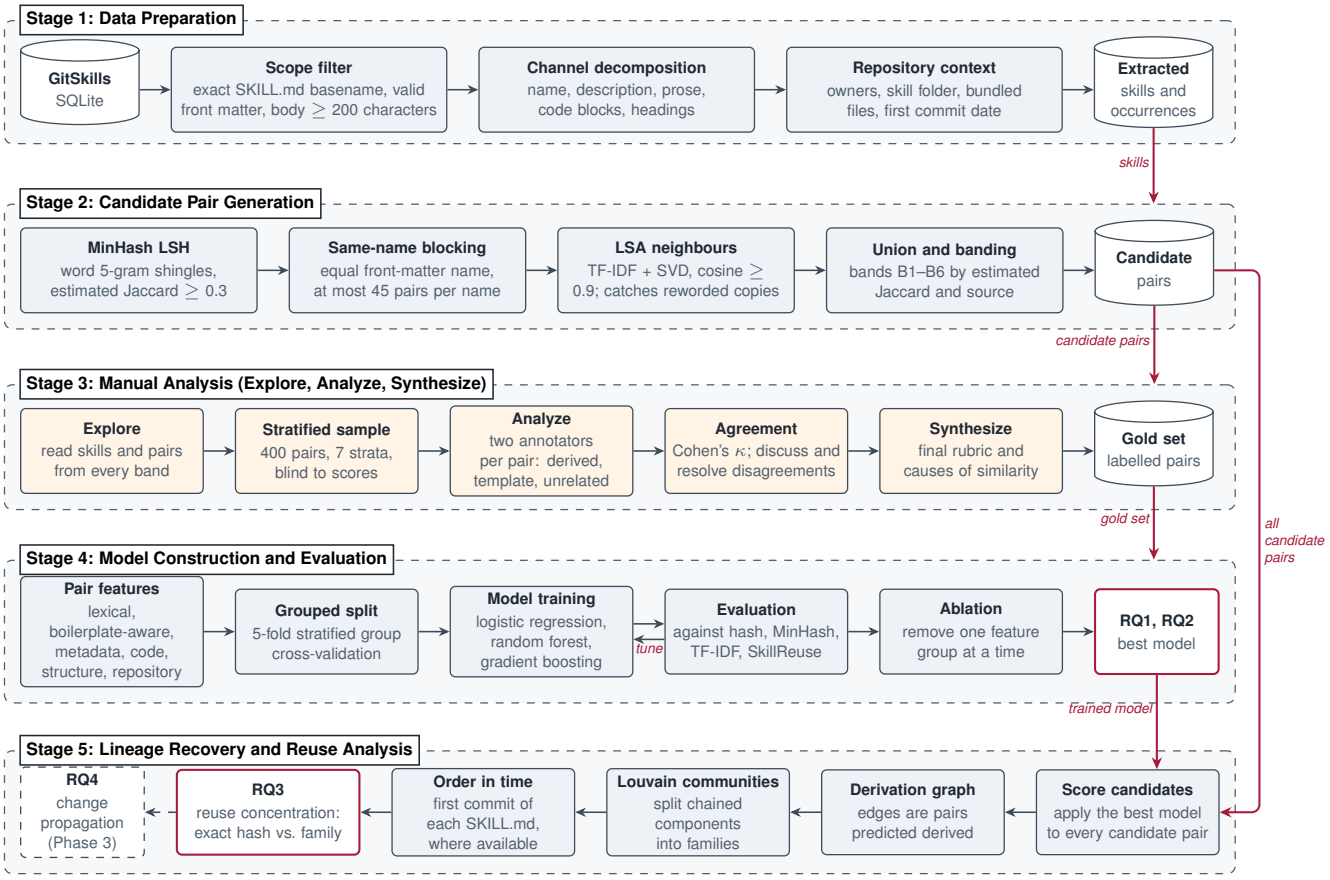


Figure 1: Approach overview. Orange boxes are manual steps; boxes with a red outline answer our research questions. Stage 5 reads right to left, because it starts from the trained model and the full set of candidate pairs.

it is *unrelated*; sharing a topic alone is not derivation. We measured agreement with Cohen's κ [5] and resolved disagreements in discussion.

Synthesise. We revised the rubric with the cases that caused disagreement and grouped the reasons pairs looked similar into a short list of causes **[list your categories, e.g., copy-and-edit, generator template, catalog house style, one author's personal template, translation]**. The resulting gold set contains **[ResGoldPairs]** pairs: **[ResGoldDerived]** derived, **[ResGoldTemplate]** template, and **[ResGoldUnrelated]** unrelated.

Stage 4: Model construction and evaluation. For each pair we compute 19 features in six groups (Table 4). The boilerplate-aware group is specific to our problem. We normalise every line of a skill (lower case, list markers removed, whitespace collapsed; at least 20 characters containing a letter) and call a line *boilerplate* when it appears in skills carrying at least **[ResBoilerK]** different front-matter names; the working set contains **[ResBoilerLines]** such lines. A widely copied skill usually keeps its name, so its lines stay distinctive, while scaffold lines recur under many unrelated names. Derived variants should therefore share distinctive lines, and template siblings mostly boilerplate. We train logistic regression, random forest, and gradient boosting classifiers with balanced class weights and evaluate them with five-fold stratified group cross-validation, so that no skill appears in both a training and a test fold (Section 7.2).

Stage 5: Lineage recovery and reuse analysis. We train the best model from Stage 4 on all gold pairs and apply it to every candidate pair. Pairs predicted as derived become edges of a derivation graph. Instead of taking connected components, which chain unrelated skills together through a few bridging edges, we split the graph into families with Louvain community detection [2]. Where both skills of a derived pair have commit history, the earlier first commit of `SKILL.md` gives the pair a direction. Finally, we compare how concentrated reuse is when counted by exact hash and by family (RQ3). Phase 3 will use the families to study change propagation (RQ4).

7 EXPERIMENTS

7.1 Research Questions

Three research questions assess our solution; a fourth is planned for Phase 3.

Table 4: Pair features used by our classifier.

Group	Features
Lexical	Jaccard and containment of word 5-gram shingles; TF-IDF cosine of the bodies; normalised edit similarity of the first 5,000 characters
Boilerplate-aware	Jaccard and containment of distinctive lines; Jaccard of boilerplate lines; share of shared lines that are boilerplate
Metadata	names equal; name similarity; TF-IDF cosine of the descriptions
Code	Jaccard of identifier tokens in code blocks; both skills contain code
Structure	similarity of heading sequences; length ratio; heading-count ratio
Repository	a shared owner; same skill-folder name; Jaccard of bundled file names

RQ1 (Effectiveness). How accurately does our approach identify derived skill variants, compared with exact hashing, similarity thresholds, and SkillReuse? This question tests whether lineage can be recovered from content at all, and whether our design improves on existing detectors on the pairs that matter most: highly similar pairs that are not derived. We report precision, recall, F1, and the Matthews correlation coefficient (MCC) [4] for the *derived* class, computed on pooled out-of-fold predictions, together with the standard deviation of F1 across folds. We also report the false-positive rate on high-similarity pairs (non-derived pairs in bands B1–B3), precision and recall per band, and, for our approach, a three-class confusion matrix and macro-F1.

RQ2 (Contributing signals). Which signals contribute most to separating derived variants from template siblings and unrelated skills? We remove one feature group at a time, retrain the best model, and report the change in F1 and in the high-similarity false-positive rate. We also train a model on lexical features only.

RQ3 (Reuse concentration). How concentrated is skill reuse when copies are grouped into derivation families rather than by exact content hash? For both units, exact hash and family, we report the number of units, the share of units with a single file occurrence, the Gini coefficient of occurrences per unit, the share of occurrences held by the top 1% of units, and the share of units and occurrences spanning two or more owners. We also report the inflation ratio (distinct contents per family), the size of the largest family, and the share of predicted derivations that commit history can orient in time.

RQ4 (Propagation, Phase 3). When an original skill changes, do its derived variants follow the change, stay frozen at an older version, or diverge?

7.2 Experimental Setup

All experiments use the **[ResGoldPairs]** gold pairs. We use five-fold stratified group cross-validation; groups are formed by **[ResGrouping]**, so no skill contributes to both training and testing. We compare four baselines with our approach.

Normalised-text hash predicts *derived* only when the two bodies are identical after lower-casing and collapsing whitespace, which is what exact hashing sees once trivial formatting is ignored. *Shingle Jaccard (MinHash)* thresholds the exact Jaccard similarity of word 5-gram shingles, the quantity MinHash estimates. *TF-IDF cosine* thresholds the cosine similarity of TF-IDF vectors of the bodies. For both, the threshold that maximises F1 is chosen on the training folds only. *SkillReuse-style* is our re-implementation of the eight features of Zhu et al. [16]: a flat TF-IDF similarity, LSA cosine similarities for the YAML, prose, and code channels (set to zero when a channel is missing), their three pairwise products, and a structural similarity over eight document statistics, combined by a class-balanced logistic regression whose threshold is again tuned on the training folds. It is trained on our gold set, so it is compared with our approach on equal terms; it is not the authors' released implementation.

Our approach uses the 19 features of Table 4 with three classifiers from scikit-learn: logistic regression on standardised features, a random forest with 500 trees, and histogram-based gradient boosting, all with balanced class weights and no threshold tuning. The classifiers are trained on the three classes when the gold set has at least 15 template pairs, and on derived versus not derived otherwise.

7.3 RQ1: Effectiveness

Table 5 compares all methods. The best configuration of our approach, **[ResOursName]**, reaches a precision of **[ResOursP]**, a recall of **[ResOursR]**, and an F1 of **[ResOursF]** (MCC **[ResOursMCC]**) on the derived class. The strongest baseline, **[ResBaseName]**, reaches an F1 of **[ResBaseF]**, and the SkillReuse-style detector **[ResSRF]**. On high-similarity pairs that annotators judged not derived, our approach labels **[ResOursHiFP]** as derived, against **[ResBaseHiFP]** for the strongest baseline. Across the three classes it reaches a macro-F1 of **[ResMacroF]** (Table 7). Table 6 shows where the differences arise. **[Two or three sentences of error analysis from results/rq1_oof_predictions.csv: what do the false positives and false negatives of the best model look like?]**

> **Answer to RQ1: [One or two sentences summarising the result.]**

7.4 RQ2: Contributing Signals

Table 8 reports the ablation. Removing the **[ResTopAblation]** group causes the largest drop in F1 (**[ResTopAblationDrop]**). **[Report what happens to the high-similarity false-positive rate when the boilerplate-aware group is removed; this is the direct test of our design choice. Also compare with the lexical-only model.]**

> **Answer to RQ2: [One or two sentences.]**

7.5 RQ3: Reuse Concentration

Scoring all **[ResCandPairs]** candidate pairs yields **[ResDerivedEdges]** predicted derivations. Grouping the **[ResWorkHashes]** distinct contents of the working set into families leaves **[ResFamilies]** units, an inflation ratio of **[ResInflation]**. **[ResMultiFamilies]** families contain two or more distinct contents, and the largest contains **[ResLargestFamily]**, compared with a largest connected

Table 5: RQ1: detection of derived variants (pooled out-of-fold predictions; F1 ± standard deviation across folds). The last column is the share of non-derived pairs in bands B1–B3 that a method labels as derived (lower is better).

[Run code/06_rq1_rq2.py to generate this table.]

Table 6: RQ1 by similarity band: precision (P) and recall (R) of the strongest baseline and of our best model.

[Run code/06_rq1_rq2.py to generate this table.]

Table 7: RQ1: confusion matrix of our best model (out-of-fold).

[Run code/06_rq1_rq2.py to generate this table.]

Table 8: RQ2: ablation of feature groups for our best model.

[Run code/06_rq1_rq2.py to generate this table.]

Table 9: RQ3: reuse concentration by exact hash and by derivation family. The last column covers the whole in-scope population of the sample at the exact-hash level.

[Run code/07_rq3.py to generate this table.]

component of **[ResLargestCC]** before community detection. The Gini coefficient of occurrences per unit moves from **[ResGiniHash]** for exact hashes to **[ResGiniFam]** for families, and the share of units spanning two or more owners from **[ResCrossHash]** to **[ResCrossFam]** (Table 9). Only **[ResOrient]** of the predicted derivations can be oriented in time from the dataset's commit history.

These figures are a lower bound. The official sample draws distinct contents uniformly, so most members of a large family fall outside it and families appear smaller than they are. Phase 3 repeats the analysis on the full dataset.

Answer to RQ3: [One or two sentences.]

8 LIMITATIONS AND NEXT STEPS

Labels. Whether two skills are derived is a judgement. The boundary between a derived variant and a template sibling is least clear for a series of skills written by one author from a personal template. We reduce this risk with a written rubric, independent double labelling, an agreement measure, and discussion of every disagreement, but the gold set still reflects our reading of the rubric.

Sampling and features. The labelled pairs are stratified by similarity, so class proportions in the gold set do not reflect proportions in the population; we therefore also report results per band. The boilerplate threshold is a heuristic, and a widely copied skill that is often renamed can push its own lines over it. The same-owner feature may let the model learn authors' habits rather than derivation; the ablation shows how much it contributes.

Baseline. Our SkillReuse baseline is re-implemented from the paper and trained on our gold set. Its numbers are not comparable to those reported by Zhu et al. [16].

Data. GitSkills covers default branches of public repositories only and is itself a lower bound on the population [8]. Because the official sample draws distinct contents uniformly, families are truncated, so RQ3 underestimates family-level reuse. Commit history is available for only 2,567 of the 11,009 skills in our working set, and `first_commit_at` dates the file at its current path, so a renamed or moved skill looks newer than it is [7].

Phase 3. We will (i) run the pipeline on the full GitSkills dataset; (ii) answer RQ4 by retrieving, for the largest families, the history of the earliest member through the GitHub API, hashing each past version, and checking whether copies match the current version, an older version, or neither, which separates consistent from inconsistent change propagation [10]; (iii) fetch commit dates for pairs the dataset cannot orient; (iv) add multilingual sentence embeddings to recover reworded and translated copies; and (v) compare against SkillReuse's released implementation and on SkillReuse-Bench.

REFERENCES

[1] Agent Skills. 2025. Agent Skills Specification. https://agentskills.io.

[2] Vincent D. Blondel, Jean-Loup Guillaume, Renaud Lambiotte, and Etienne Lefebvre. 2008. Fast Unfolding of Communities in Large Networks. *Journal of Statistical Mechanics: Theory and Experiment* 2008, 10 (2008), P10008. https://doi.org/10.1088/1742-5468/2008/10/P10008

[3] Andrei Z. Broder. 1997. On the Resemblance and Containment of Documents. In *Proceedings of Compression and Complexity of Sequences*. 21–29. https://doi.org/10.1109/SEQUEN.1997.666900

[4] Davide Chicco and Giuseppe Jurman. 2020. The Advantages of the Matthews Correlation Coefficient (MCC) over F1 Score and Accuracy in Binary Classification Evaluation. *BMC Genomics* 21 (2020), 6. https://doi.org/10.1186/s12864-019-6413-7

[5] Jacob Cohen. 1960. A Coefficient of Agreement for Nominal Scales. *Educational and Psychological Measurement* 20, 1 (1960), 37–46. https://doi.org/10.1177/001316446002000104

[6] Scott Deerwester, Susan T. Dumais, George W. Furnas, Thomas K. Landauer, and Richard Harshman. 1990. Indexing by Latent Semantic Analysis. *Journal of the American Society for Information Science* 41, 6 (1990), 391–407.

[7] Giuseppe Destefanis, Daniel Graziotin, Matteo Vaccargiu, and Marco Ortu. 2026. GitSkills: Sample for the MSR 2027 Mining Challenge Proposal. https://github.com/giuseppedestefanis/gitskills-sample.

[8] Giuseppe Destefanis, Daniel Graziotin, Matteo Vaccargiu, and Marco Ortu. 2027. GitSkills: A Dataset of Agent Skills on GitHub. In *Proceedings of the 24th International Conference on Mining Software Repositories (MSR '27)*. ACM. To appear. arXiv:2608.10906.

[9] Haoyu Gao, Jai Lal Lulla, Hong Yi Lin, Sebastian Baltes, Christoph Treude, and Mansooreh Zahedi. 2026. From Registry to Repository: How AI Agent Skills Are Written, Adapted, and Maintained. arXiv:2607.00911.

[10] Miryung Kim, Vibha Sazawal, David Notkin, and Gail C. Murphy. 2005. An Empirical Study of Code Clone Genealogies. In *Proceedings of the 10th European Software Engineering Conference held jointly with the 13th ACM SIGSOFT International Symposium on Foundations of Software Engineering (ESEC/FSE)*. https://doi.org/10.1145/1095430.1081737

[11] Yi Liu, Weizhe Wang, Ruitao Feng, Yao Zhang, Guangquan Xu, Gelei Deng, Yuekang Li, and Leo Zhang. 2026. Agent Skills in the Wild: An Empirical Study of Security Vulnerabilities at Scale. arXiv:2601.10338.

[12] Cristina V. Lopes, Petr Maj, Pedro Martins, Vaibhav Saini, Di Yang, Jakub Zitny, Hitesh Sajnani, and Jan Vitek. 2017. DéjàVu: A Map of Code Duplicates on GitHub. *Proceedings of the ACM on Programming Languages* 1, OOPSLA, Article 84 (2017). https://doi.org/10.1145/3133908

[13] Chanchal K. Roy, James R. Cordy, and Rainer Koschke. 2009. Comparison and Evaluation of Code Clone Detection Techniques and Tools: A Qualitative Approach. *Science of Computer Programming* 74, 7 (2009), 470–495. https://doi.org/10.1016/j.scico.2009.02.007

[14] Hitesh Sajnani, Vaibhav Saini, Jeffrey Svajlenko, Chanchal K. Roy, and Cristina V. Lopes. 2016. SourcererCC: Scaling Code Clone Detection to Big-Code. In *Proceedings of the 38th International Conference on Software Engineering (ICSE)*. 1157–1168. https://doi.org/10.1145/2884781.2884877

[15] Xinjian Wu, Jingzhi Gong, Gunel Jahangirova, Zhenpeng Chen, and Jie M. Zhang. 2026. An Empirical Study of Downstream Adaptation for Agent Skills. arXiv:2607.03238.

[16] Jiaying Zhu, Lyuye Zhang, Wenbo Guo, and Yang Liu. 2026. Latent Reuse in Agent Skills: Multi-modal Clone Detection at Ecosystem Scale. In *Proceedings of the 41st IEEE/ACM International Conference on Automated Software Engineering (ASE '26)*. https://doi.org/10.1145/3832783.3837468